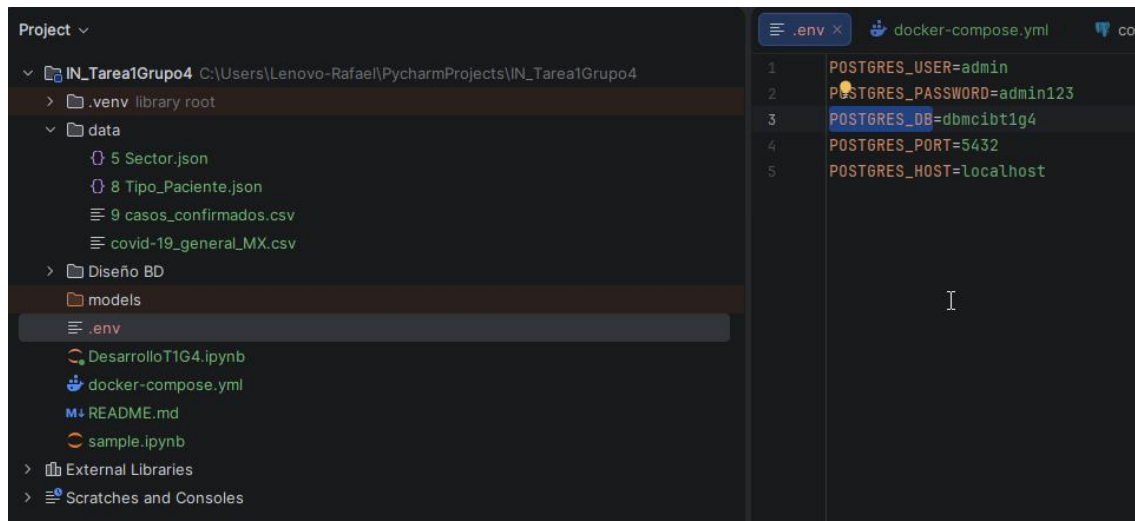


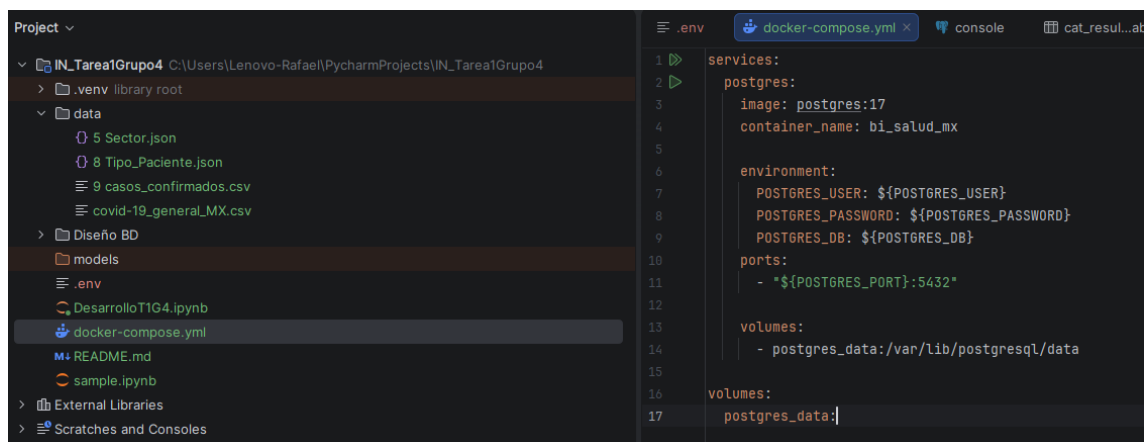
1. Creación de Docker

1.1. Mediante un docker-compose deberás utilizar el conocimiento adquirido y empezar a gestionar dicho contenedor toma en cuenta las credenciales de la base de datos deben estar en un archivo .env

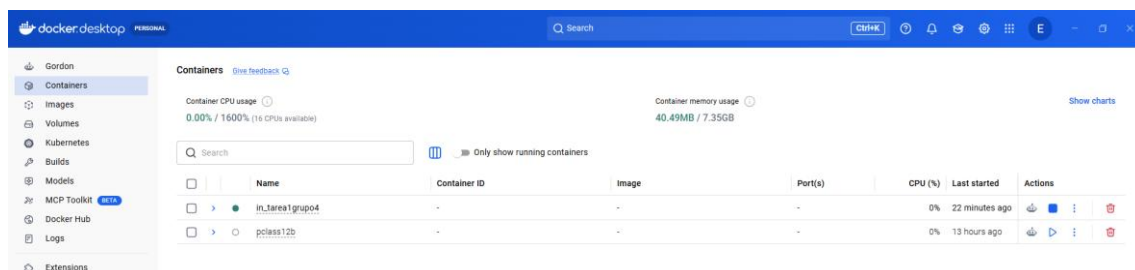
Variables .env



Activación de Docker Compose

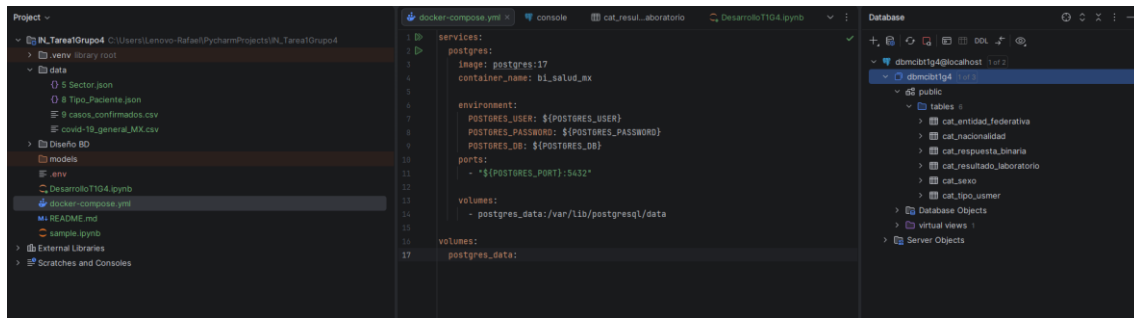


Docker Desktop Online

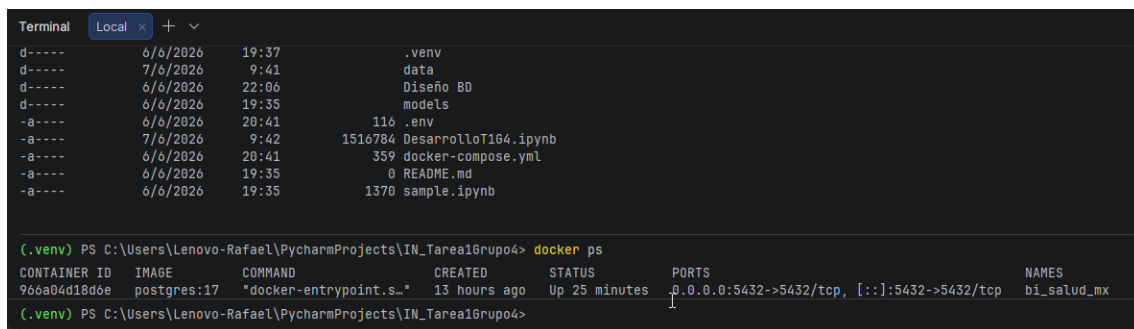


1.2. Aquí se deberá alojar una base de datos en postgres sobre la cual se va a cargar la información

Base de Datos Postgres 17

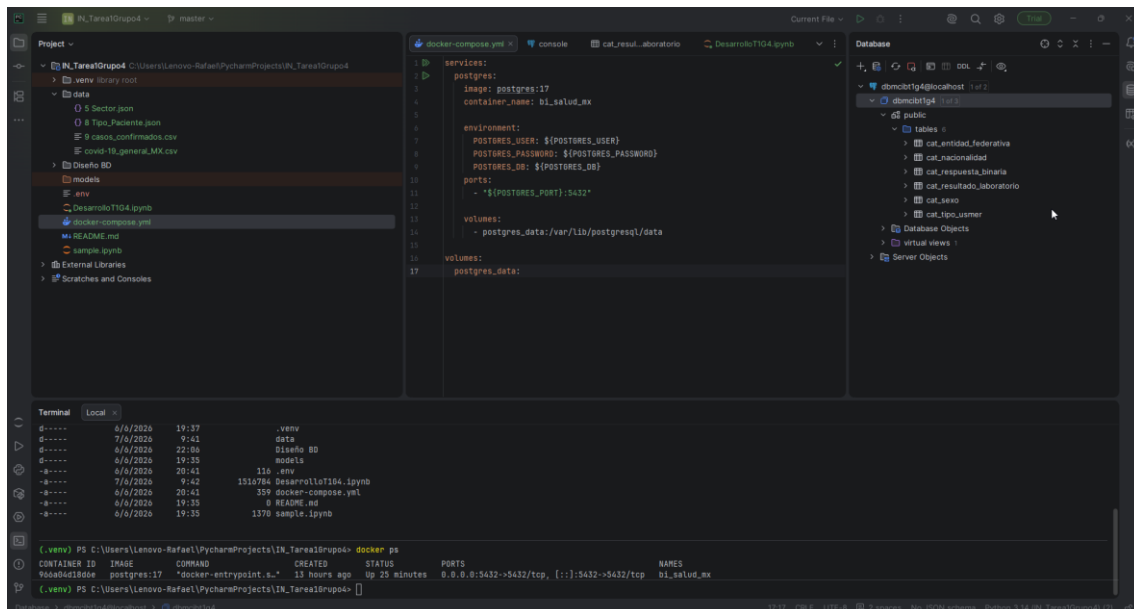


- 1.3. Adjuntar una captura de pantalla donde se visualice la ejecución exitosa del comando de ejecución del compose y los contenedores activos mediante el comando correspondiente.

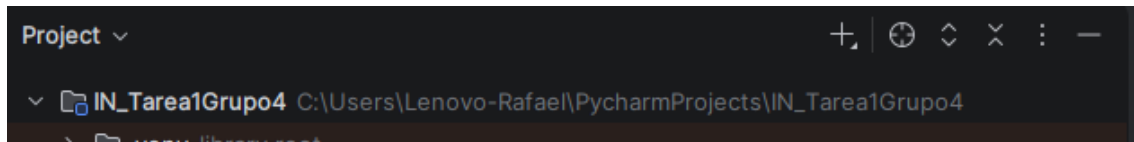


2. Descarga y configura Pycharm-Pro de JetBrains

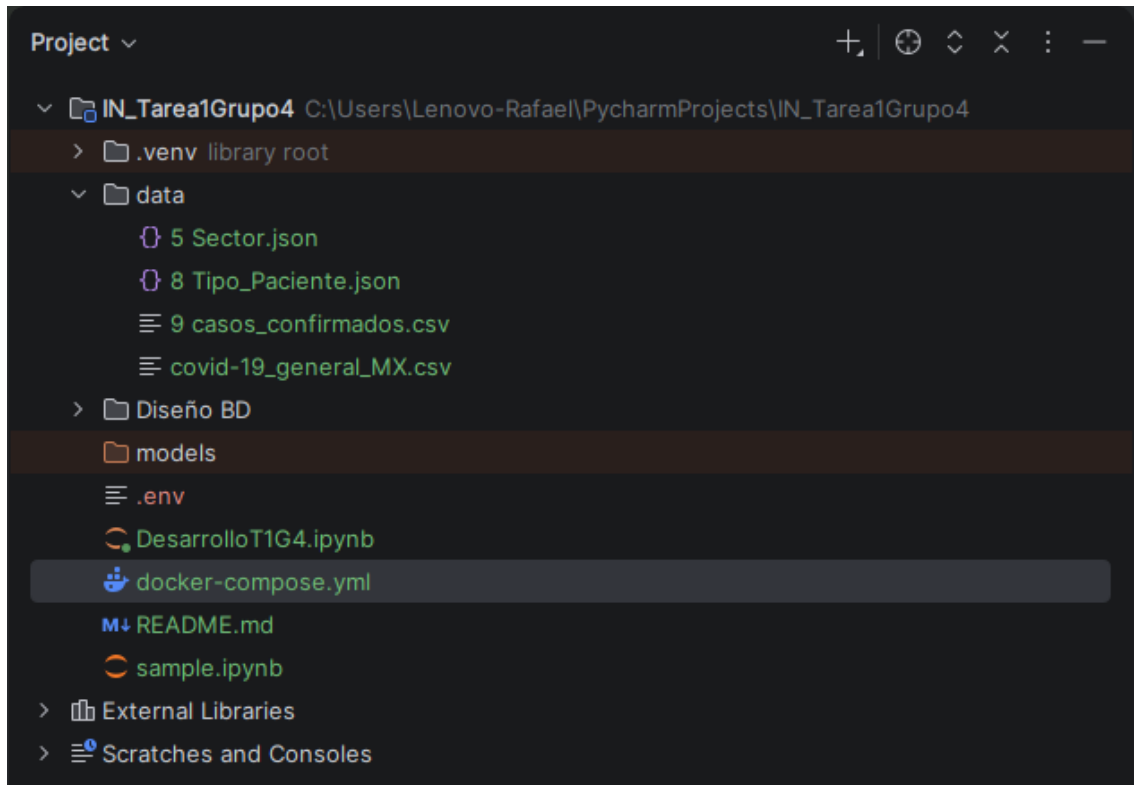
- 2.1. Pycharm-Pro es un IDE completo para análisis de datos, además de conectarte y hacer consultas a bases de datos, se podrá ejecutar Jupyter Notebooks.



- 2.2. Crea un nuevo proyecto llamado “Practica_G#”

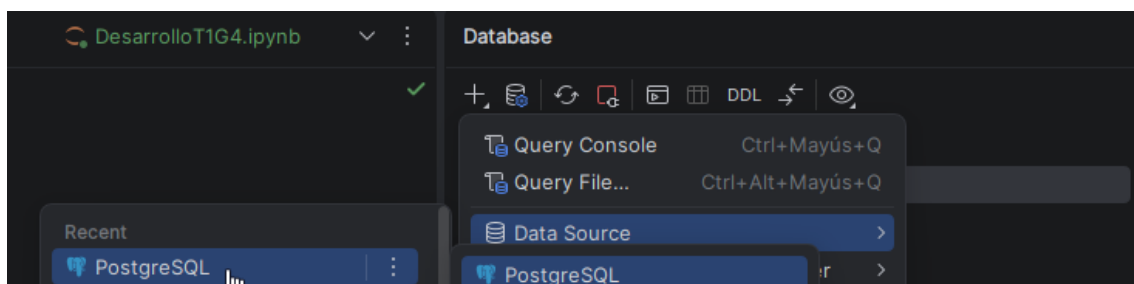


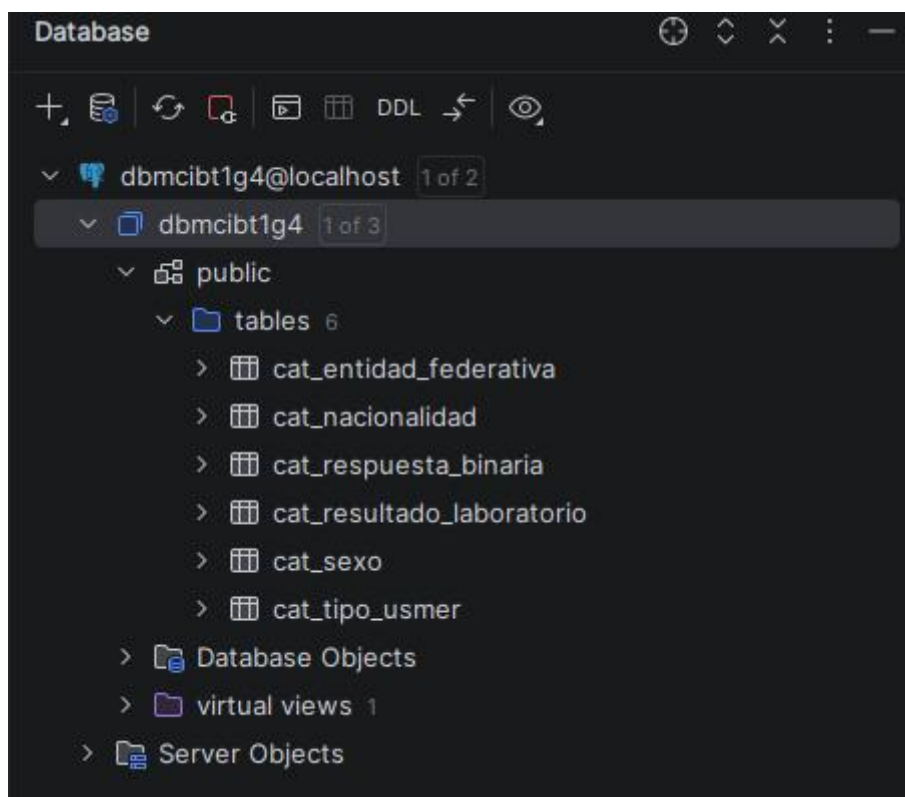
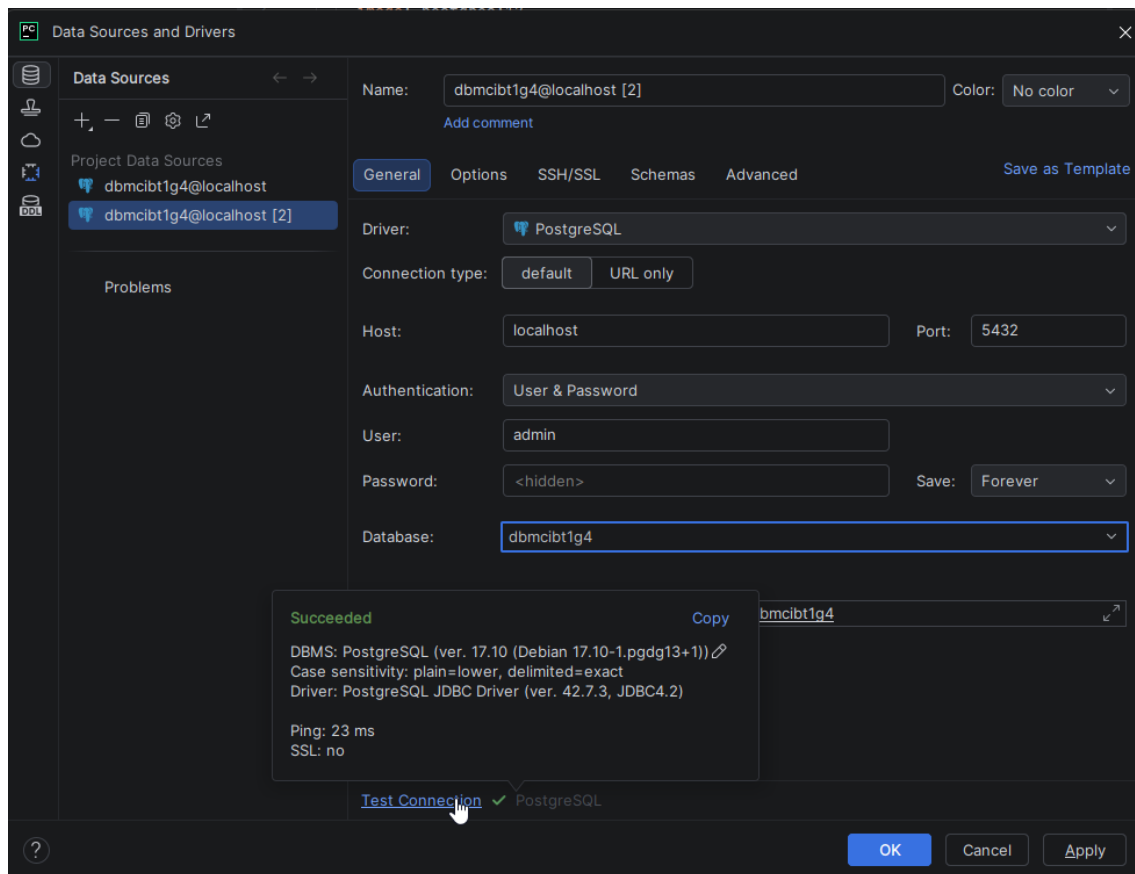
2.3. Crea un environment este será el ambiente que se utilice dentro de esta práctica



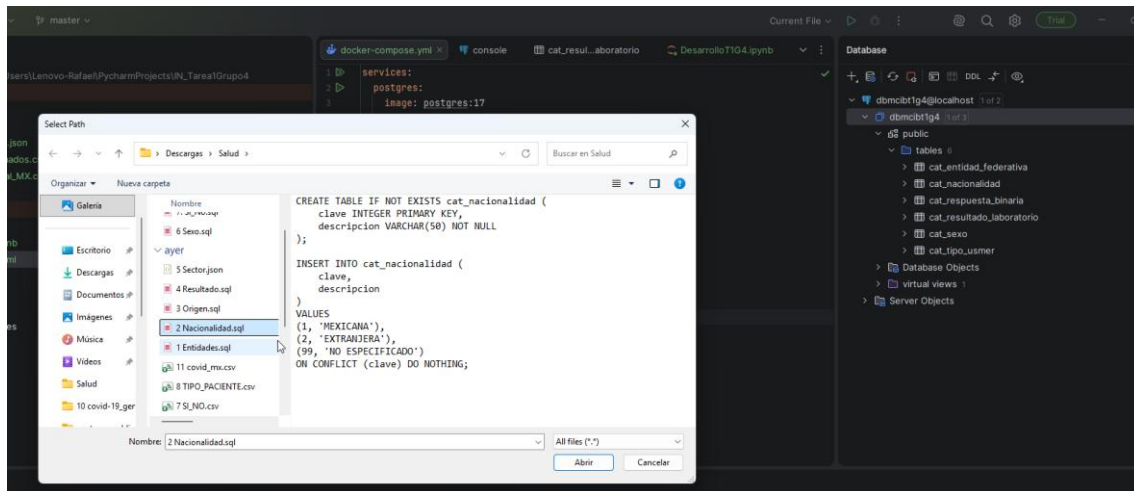
3. Crea una conexión a la base de datos de PostgreSQL

3.1. En el IDE crea la conexión a la base de datos





- 3.2. **Carga la base de datos:** esta información puede ser migrada desde un script de pandas o se puede generar un script para generarla y cargarla de datos.



4. Genera los tres DataFrames principales en función a cada una de las fuentes de datos con las que se va a trabajar

4.1. Explora y entiende los datos de tus distintos DataFrames

Carga de Información a la BD PostGres 17

engine

`create_engine(f'postgresql+psycopg2://{DB_USER}:{DB_PASSWORD}@{DB_HOST}/{DB_NAME}')`

```

1 df_entidades = pd.read_sql('select * from cat_entidad_federativa', engine)
✓ [4] 84ms

1 df_nacionalidad = pd.read_sql('select * from cat_nacionalidad', engine)
✓ [5] 21ms

1 df_origen = pd.read_sql('select * from cat_tipo_usmer', engine)
✓ [6] 16ms

1 df_resultado = pd.read_sql('select * from cat_resultado_laboratorio', engine)
✓ [7] 27ms

1 df_sexo = pd.read_sql('select * from cat_sexo', engine)
✓ [18] 21ms

1 df_respuesta = pd.read_sql('select * from cat_respuesta_binaria', engine)
✓ [19] 27ms

```

Carga de Datos Tipo Json

```

1 df_sector = pd.read_json("data/5 Sector.json")
✓ [8] 56ms

1 df_tipopaciente = pd.read_json('data/5 Sector.json')
✓ [20] 22ms

```

Carga de Datos Tipo CSV

```
1 df_covidgeneral = pd.read_csv("data/covid-19_general_MX.csv")
✓ [11] 1s 402ms
```

```
1 df_casosconfirmados = pd.read_csv('data/9 casos_confirmados.csv')
✓ [21] 212ms
```

4.2. Genera tres Scripts de análisis de datos a cada uno de los DataFrames

- Mostrar las primeras 5 filas de cada uno.

Postgres 17

```
1 df_entidades = pd.read_sql('select * from cat_entidad_federativa limit 5',
2 df_entidades
engine)
✓ [29] 34ms
```

5 rows 5 rows x 3 cols			
	clave_entidad	entidad_federativa	abreviatura
0	1	AGUASCALIENTES	AS
1	2	BAJA CALIFORNIA	BC
2	3	BAJA CALIFORNIA SUR	BS
3	4	CAMPECHE	CC
4	5	COAHUILA DE ZARAGOZA	CL

```
1 df_nacionalidad = pd.read_sql('select * from cat_nacionalidad limit 5', engine)
2 df_nacionalidad
✓ [30] 32ms
```

3 rows 3 rows x 2 cols		
	clave	descripcion
0	1	MEXICANA
1	2	EXTRANJERA
2	99	NO ESPECIFICADO

```
1 df_origen = pd.read_sql('select * from cat_tipo_usmer limit 5', engine)
2 df_origen
✓ [31] 30ms
```

÷	123 clave	÷	≡ descripcion	÷
0		1	USMER	
1		2	FUERA DE USMER	
2		99	NO ESPECIFICADO	

```
1 df_resultado = pd.read_sql('select * from cat_resultado_laboratorio limit 5',
2 df_resultado
✓ [32] 28ms
```

÷	123 clave	÷	≡ descripcion	÷
0		1	Positivo SARS-CoV-2	
1		2	No positivo SARS-CoV-2	
2		3	Resultado pendiente	

```
1 df_sexo = pd.read_sql('select * from cat_sexo limit 5', engine)
2 df_sexo
✓ [33] 32ms
```

÷	123 clave	÷	≡ descripcion	÷
0		1	MUJER	
1		2	HOMBRE	
2		99	NO ESPECIFICADO	

```
1 df_respuesta = pd.read_sql('select * from cat_respuesta_binaria limit 5', engine)
2 df_respuesta
✓ [34] 41ms
```

÷	123 clave	÷	≡ descripcion	÷
0		1	SI	
1		2	NO	
2		97	NO APLICA	
3		98	SE IGNORA	
4		99	NO ESPECIFICADO	

Formato Json

```
1 df_sector = pd.read_json("data/5 Sector.json")
2 df_sector.head(5)
```

✓ [35] 28ms

	clave	descripcion
0	1	CRUZ ROJA
1	2	DIF
2	3	ESTATAL
3	4	IMSS
4	5	IMSS-BIENESTAR

```
1 df_tipopaciente = pd.read_json('data/5 Sector.json')
2 df_tipopaciente.head(5)
```

✓ [37] 28ms

	clave	descripcion
0	1	CRUZ ROJA
1	2	DIF
2	3	ESTATAL
3	4	IMSS
4	5	IMSS-BIENESTAR

Formato CVS

```
1 df_covidgeneral = pd.read_csv("data/covid-19_general_MX.csv")
2 df_covidgeneral.head(5)
```

✓ [38] 1s 275ms

	Unnamed: 0	SECTOR	ENTIDAD_UM	SEXO	ENTIDAD_RES
0	0	4	9	2	
1	1	4	9	2	
2	2	4	8	1	
3	3	4	30	1	
4	4	3	15	2	


```
1 df_casosconfirmados = pd.read_csv('data/9 casos_confirmados.csv') df_casosconfir
2 df_casosconfirmados.head(5)
✓ [39] 186ms
```

Unnamed: 0	State	Sex	Age	Date
0	distrito federal	MASCULINO	28	2020-0
1	distrito federal	MASCULINO	49	2020-0
2	chihuahua	FEMENINO	67	2020-0
3	veracruz de ignacio de la lla...	FEMENINO	41	2020-0
4	méxico	MASCULINO	43	2020-0

- Identificar qué columnas tienen valores nulos.

```
1 df_entidades = pd.read_sql('select * from cat_entidad_federativa', engine)
2 df_entidades.isnull().values.any()
✓ [40] 52ms
np.False_

1 df_nacionalidad = pd.read_sql('select * from cat_nacionalidad', engine)
2 df_nacionalidad.isnull().values.any() df_nacionalidad
3
✓ [41] 44ms
np.False_

1 df_origen = pd.read_sql('select * from cat_tipo_usmer', engine)
2 df_origen.isnull().values.any()
✓ [42] 42ms
np.False_
```

```
1 df_resultado = pd.read_sql('select * from cat_resultado_laboratorio', engine)
2 df_resultado.isnull().values.any()
✓ [43] 34ms

np.False_

1 df_sexo = pd.read_sql('select * from cat_sexo', engine)
2 df_sexo.isnull().values.any()
✓ [44] 48ms

np.False_

1 df_respuesta = pd.read_sql('select * from cat_respuesta_binaria', engine)
2 df_respuesta.isnull().values.any()
✓ [46] 22ms

np.False_

Code Markdown AI SQL

1 df_sector = pd.read_json("data/5 Sector.json")
2 df_sector.isnull().values.any()
✓ [47] 42ms

np.False_
```

```
1 df_tipopaciente = pd.read_json('data/5 Sector.json')
2 df_tipopaciente.isnull().values.any()
✓ [48] 34ms

np.False_

1 df_covidgeneral = pd.read_csv("data/covid-19_general_MX.csv")
2 df_covidgeneral.isnull().values.any()
✓ [49] 1s 446ms

np.False_

1 df_casosconfirmados = pd.read_csv('data/9 casos_confirmados.csv')
2 df_casosconfirmados.isnull().values.any()
✓ [50] 244ms

np.False_
```

- Contar cuántos valores faltan en cada columna.

```
1 df_entidades = pd.read_sql('select * from cat_entidad_federativa', engine)
2 df_entidades.isnull().sum()
✓ [51] 32ms
```

	123 <unnamed>
clave_entidad	0
entidad_federativa	0
abreviatura	0

```
1 df_nacionalidad = pd.read_sql('select * from cat_nacionalidad', engine)
2 df_nacionalidad.isnull().sum()
✓ [52] 31ms
```

	123 <unnamed>
clave	0
descripcion	0

```
1 df_origen = pd.read_sql('select * from cat_tipo_usmer', engine)
2 df_origen.isnull().sum()
✓ [53] 40ms
```

	123 <unnamed>
clave	0
descripcion	0

```
1 df_resultado = pd.read_sql('select * from cat_resultado_laboratorio', engine)
2 df_resultado.isnull().sum()
✓ [54] 40ms
```

	123 <unnamed>	
clave		0
descripcion		0

```
1 df_sexo = pd.read_sql('select * from cat_sexo', engine)
2 df_sexo.isnull().sum()
✓ [56] 21ms
```

	123 <unnamed>	
clave		0
descripcion		0

```
1 df_respuesta = pd.read_sql('select * from cat_respuesta_binaria', engine)
2 df_respuesta.isnull().sum()
✓ [57] 34ms
```

	123 <unnamed>	
clave		0
descripcion		0

```
1 df_sector = pd.read_json("data/5 Sector.json")
2 df_sector.isnull().sum()
✓ [58] 23ms
```

	123 <unnamed>	
clave		0
descripcion		0

```
1 df_tipopaciente = pd.read_json('data/5 Sector.json')
2 df_tipopaciente.isnull().sum()
✓ [59] 23ms
```

	123 <unnamed>
clave	0
descripcion	0

```
1 df_covidgeneral = pd.read_csv("data/covid-19_general_MX.csv")
2 df_covidgeneral.isnull().sum()
✓ [60] 1s 247ms
```

	123 <unnamed>
Unnamed: 0	0
SECTOR	0
ENTIDAD_UM	0
SEXO	0
ENTIDAD_RES	0
TIPO_PACIENTE	0
FECHA_INGRESO	0
FECHA_SINTOMAS	0
FECHA_DEF	0
INTUBADO	0

```
1 df_casosconfirmados = pd.read_csv('data/9 casos_confirmados.csv')
2 df_casosconfirmados.isnull().sum()
✓ [61] 218ms
```

	123 <unnamed>
Unnamed: 0	0
State	0
Sex	0
Age	0
Date	0
Confirmed	0

- Calcular el promedio, máximo y mínimo de una variable numérica a su elección.

```

1 df_covidgeneral = pd.read_csv("data/covid-19_general_MX.csv")
2 print(f"{df_covidgeneral["EDAD"].mean():.2f}")
3
✓ [70] 1s 164ms
42.55

1 df_covidgeneral = pd.read_csv("data/covid-19_general_MX.csv")
2 print(df_covidgeneral["EDAD"].max())
✓ [67] 1s 210ms
120

1 df_covidgeneral = pd.read_csv("data/covid-19_general_MX.csv")
2 print(df_covidgeneral["EDAD"].min())
✓ [68] 1s 186ms
0

```

- Encontrar el valor máximo y mínimo de dos variables agrupadas.

```

1 df_casosconfirmados.groupby("State")["Age"].agg(["min", "max"])
✓ [78] 48ms

```

State	min	max
aguascalientes	0	99
baja california	0	99
baja california sur	0	100
campeche	1	100
chiapas	0	106
chihuahua	0	100
coahuila	0	95
colima	0	96
distrito federal	0	105
durango	0	105

```

1 df_casosconfirmados.groupby("Sex")["Age"].agg(["min", "max", "mean"])
✓ [81] 47ms

```

Sex	min	max	mean
FEMENINO	0	105	44.539639
MASCULINO	0	119	45.998044

